

Prctica 11

Cesar zaid reynoso mtz

Jesus collazo

Programacion orientada a objetos

INTRODUCCION

Que es la herencia?

La herencia es un concepto fundamental de la Programación Orientada a Objetos (POO) que permite crear nuevas clases a partir de una clase existente. La clase original, conocida como clase base o padre, contiene atributos y métodos comunes que pueden ser reutilizados por otras clases llamadas clases derivadas o hijas.

Este mecanismo facilita la reutilización de código, evita la duplicación y permite organizar mejor los programas, ya que se pueden definir características generales en una sola clase y luego especializarlas en otras. Además, la herencia permite que las clases derivadas modifiquen o amplíen el comportamiento de la clase base, adaptándose a diferentes necesidades.

En el desarrollo de software, la herencia es especialmente útil para modelar situaciones del mundo real donde varios objetos comparten características comunes, pero también presentan diferencias. Por ejemplo, en un sistema de personajes de un videojuego, todos pueden tener atributos como nombre, nivel y vida, pero cada tipo de personaje puede tener habilidades únicas.

ARCHIVO.H

En este módulo se definen todas las clases del programa. Aquí se establece la estructura general del sistema, incluyendo la clase base Personaje, que contiene los atributos comunes como nombre, nivel y vida, así como los métodos mostrarInfo() y atacar().

También se declaran las clases derivadas Guerrero, Mago y Arquero, las cuales heredan de Personaje. En este archivo solo se especifica qué hace cada clase, pero no cómo lo hace. Es decir, funciona como un “plano” o diseño del programa.

PERSONAJE.CPP

Se define cómo funciona mostrarInfo() para mostrar los datos del personaje, y cómo cada clase realiza su ataque. Cada clase derivada sobrescribe el método atacar(), permitiendo que cada tipo de personaje tenga un comportamiento distinto (por ejemplo, el guerrero hace un ataque fuerte, el mago usa magia y el arquero dispara flechas).

MAIN.CPP

Este módulo es el punto de entrada del programa. Aquí se crean los objetos de cada tipo de personaje (Guerrero, Mago y Arquero) y se ejecutan sus métodos.

Se muestra la información de cada personaje y se realizan sus ataques, lo que permite observar cómo funciona la herencia y el polimorfismo en la práctica.

Como se uso la herencia?

La herencia se utilizó para crear diferentes tipos de personajes a partir de una clase general. Primero se definió una clase base llamada Personaje, la cual contiene los atributos y métodos comunes como nombre, nivel, vida, mostrar información y atacar.

Después, se crearon las clases Guerrero, Mago y Arquero, las cuales heredan de Personaje usando la

sintaxis :

public Personaje. Gracias a esto, estas clases no necesitan volver a declarar los atributos y métodos básicos, ya que los obtienen automáticamente de la clase base.

Además, cada clase derivada sobrescribe el método atacar(), permitiendo que cada personaje tenga un comportamiento distinto. De esta manera, se reutiliza código y al mismo tiempo se personaliza el funcionamiento de cada tipo de personaje.

Diferencia entre clase base y la derivada

La clase base es la clase principal que contiene las características generales que serán compartidas. En este caso, Personaje es la clase base porque define los atributos comunes (nombre, nivel, vida) y los métodos generales.

Por otro lado, las clases derivadas son aquellas que se crean a partir de la clase base. Estas heredan sus características, pero también pueden modificar o ampliar su comportamiento. En el programa, Guerrero, Mago y Arquero son clases derivadas, ya que cada una adapta el método atacar() según su tipo.

CONCLUSION

En esta práctica se logró comprender y aplicar el concepto de herencia dentro de la Programación Orientada a Objetos, permitiendo estructurar un sistema de manera más organizada y eficiente. A través de la creación de una clase base y varias clases derivadas, se pudo reutilizar código y evitar la duplicación de atributos y métodos, lo que facilita el desarrollo y mantenimiento del programa.

Asimismo, se observó cómo cada clase puede adaptar su comportamiento mediante la sobrescritura de métodos, en este caso el método atacar(), lo que demuestra el uso del polimorfismo. Esto permitió que cada tipo de personaje tuviera acciones específicas sin perder la estructura común.

la herencia es una herramienta fundamental que ayuda a modelar problemas de la vida real de forma más clara, haciendo que los programas sean más escalables, reutilizables y fáciles de entender. Esta práctica permitió reforzar estos conocimientos y comprender mejor su aplicación en el desarrollo de software.

```
#ifndef PERSONAJE_H
#define PERSONAJE_H

#include <iostream>
#include <string>
using namespace std;

// Clase base
class Personaje {
protected:
    string nombre;
    int nivel;
    int vida;

public:
    Personaje(string nom, int niv, int vid);
    void mostrarInfo();
    virtual void atacar(); // método virtual
};

// Clase Guerrero
class Guerrero : public Personaje {
public:
    Guerrero(string nom, int niv, int vid);
    void atacar() override;
};

// Clase Mago
class Mago : public Personaje {
public:
    Mago(string nom, int niv, int vid);
    void atacar() override;
};

// Clase Arquero
class Arquero : public Personaje {
public:
```

```

#ifndef PERSONAJE_H
#define PERSONAJE_H

#include <iostream>
#include <string>
using namespace std;

// Clase base
class Personaje {
protected:
    string nombre;
    int nivel;
    int vida;

public:
    Personaje(string nom, int niv, int vid);
    void mostrarInfo();
    virtual void atacar(); // método virtual
};

// Clase Guerrero
class Guerrero : public Personaje {
public:
    Guerrero(string nom, int niv, int vid);
    void atacar() override;
};

// Clase Mago
class Mago : public Personaje {
public:
    Mago(string nom, int niv, int vid);
    void atacar() override;
};

// Clase Arquero
class Arquero : public Personaje {
public:

```

```

#ifndef PERSONAJE_H
#define PERSONAJE_H

#include <iostream>
#include <string>
using namespace std;

// Clase base
class Personaje {
protected:
    string nombre;
    int nivel;
    int vida;

public:
    Personaje(string nom, int niv, int vid);
    void mostrarInfo();
    virtual void atacar(); // método virtual
};

// Clase Guerrero
class Guerrero : public Personaje {
public:
    Guerrero(string nom, int niv, int vid);
    void atacar() override;
};

// Clase Mago
class Mago : public Personaje {
public:
    Mago(string nom, int niv, int vid);
    void atacar() override;
};

// Clase Arquero
class Arquero : public Personaje {
public:

```